RAPPORT TECHNIQUE DE VALIDATION DE SIMULATION

Validation du Jalon AMR : Déploiement Gazebo 11, RViz 2 et Téléopération Clavier (/cmd_vel)
Résolution des Incidents Graphiques WSLg & Architecture des Flux de Données

Master Professionnel RAIA • ISET Bizerte | Réf. Mémoire : RAIA 01.26 | Encadrement : M. Hmida Hmaied

1. Contexte et Validation du Jalon (Milestone)

Dans le cadre du projet de Master RAIA portant sur la modernisation du robot suiveur de ligne (AGV) issu du rapport **RAIA 01.26** en un **AMR (Autonomous Mobile Robot)**, une étape charnière a été franchie avec succès : la mise en œuvre, la communication et le pilotage temps-réel du jumeau numérique dans l'environnement de simulation 3D **Gazebo** couplé au visualiseur **RViz 2**. Le modèle 3D conçu en Xacro intègre fidèlement la mécanique de la Fig. 26 (châssis à rebords oblongs, double rack de transport de PCB, propulsion différentielle et capteur LiDAR 2D).

2. Analyse et Résolution de l'Incident Technique : Écran Noir Gazebo

Lors de l'exécution initiale de la simulation sous Windows 11 / WSLg, Gazebo s'ouvrait mais présentait une fenêtre 3D totalement noire sans charger l'atelier. L'audit technique a isolé deux causes simultanées :

• **Blocage Réseau des Modèles SDF Distants :** Au premier démarrage, Gazebo tente de télécharger les modèles du soleil (sun) et du sol (ground_plane) depuis le serveur models.gazebosim.org. Ce serveur étant fréquemment inaccessible ou sujet à de fortes latences, le thread de rendu restait figé indéfiniment.
-> **Résolution :** Création et mise en cache locale des fichiers model.config et model.sdf directement dans ~/.gazebo/models/sun/ et ~/.gazebo/models/ground_plane/.

• **Conflit de Rendu Direct3D 12 vGPU sur Puce Intel UHD :** Sous WSLg, le pilote Mesa D3D12 Gallium présente une instabilité de synchronisation des buffers d'échange OpenGL avec le moteur de rendu OGRE 1.x de Gazebo Classic.
-> **Résolution :** Configuration de la variable d'environnement GALLIUM_DRIVER=llvmpipe dans ~/.bashrc, basculant le rendu OpenGL vers le rastériseur CPU multi-cœurs LLVMpipe (parfaitement conforme OpenGL 4.5 sans freeze).

> **RÉSULTAT DU DÉPANNAGE :** L'application de ces deux correctifs a débloqué instantanément l'affichage : l'atelier d'usine (factory_workshop.world) et l'AMR apparaissent désormais en temps réel sans latence d'initialisation.

3. Décomposition et Rôle des 3 Commandes Exécutées

La chaîne de test s'articule autour de trois processus ROS 2 exécutés en parallèle dans des terminaux distincts :

Commande	Processus Lancés	Rôle et Impact dans la Simulation
`ros2 launch raia_amr_description sim.launch.py`	• gzserver • gzclient • robot_state_publisher • spawn_entity	1. Démarre le moteur physique ODE (gravité, collisions, contacts roues/sol). 2. Parse le modèle Xacro en URDF et publie la cinématique /robot_description. 3. Insère le robot AMR dans l'atelier factory_workshop à la pose (0,0,0.05). 4. Active les plugins : moteur différentiel, faisceau LiDAR et caméra.
`ros2 launch raia_amr_description rviz.launch.py`	• rviz2 • joint_state_publisher_gui	1. Ouvre l'interface de supervision et de télémétrie 3D RViz 2. 2. Affiche l'arbre des repères géométriques (/tf :

		base_footprint -> laser_frame). 3. Visualise le nuage de points LiDAR rouge (/scan) en impact sur les murs. 4. Affiche l'odométrie et la trajectoire spatiale calculée du robot.
`ros2 run teleop_twist_keyboard teleop_twist_keyboard`	• teleop_twist_keyboard	1. Intercepte les frappes clavier (touches 'i', ',', 'j', 'l', 'k'). 2. Convertit les commandes en vecteurs de vitesse (linear.x, angular.z). 3. Publie en continu sur le topic ROS 2 /cmd_vel pour piloter l'AMR.

4. Architecture des Flux et Graphe Computationnel ROS 2

Lors de la manipulation du clavier, la circulation des données suit le schéma interactif suivant :

1. Téléopération : L'utilisateur presse la touche 'i' -> teleop_twist_keyboard publie geometry_msgs/Twist (linear.x = 0.5 m/s) sur /cmd_vel.

2. Cinématique Différentielle : Le plugin libgazebo_ros_diff_drive reçoit /cmd_vel et calcule les vitesses angulaires des roues motrices gauche et droite selon l'écartement d = 0.35 m et le rayon r = 0.05 m :

```
Vitesse roue gauche (w_G) = (Vx - (d/2) * Wz) / r  |  Vitesse roue droite (w_D) = (Vx + (d/2) * Wz) / r
```

3. Simulation Physique : Gazebo applique les couples moteurs, simule le frottement au sol (mu = 1.2), déplace le châssis et met à jour l'odométrie sur /odom ainsi que la transformation tf odom -> base_footprint.

4. Perception LiDAR : Le capteur laser virtuel balaie l'espace à 10 Hz sur 360°, détecte les tables de PCB et murs de l'atelier, et publie les distances sur /scan.

5. Rendu RViz 2 : RViz reçoit /scan et /tf et affiche en temps réel le robot mobile et l'impact laser rouge sur les obstacles environnants.

5. Prochaines Étapes : Cartographie SLAM et Navigation Nav2

Le jalon de simulation dynamique étant validé, la plateforme est immédiatement prête pour les deux étapes clés de l'autonomie AMR :

- • Cartographie 2D (SLAM) : Lancement du package slam_toolbox en pilotant le robot pour enregistrer la carte globale de l'usine (carte de grille d'occupation au format .yaml et .pgm).
- • Navigation Autonome (Nav2) : Configuration du planificateur de trajectoire global (Navfn/Smac) et du contrôleur local (DWB) pour envoyer des destinations autonomes ('2D Nav Goal') dans RViz.